

Fase 02

Partimos de tu estado actual: el reporte ya indica que el núcleo de SOFIAA está vivo en localhost con IA en streaming, memoria, voz, seguridad y UI funcionales, y que el proyecto va en 78% con 4 de 5 sprints completados.

Ahora sí: **Fase 2** debe ser la fase de **robustez, inteligencia estructural y pre-deploy hardening**.

La idea es que no metamos más “features sueltas”, sino **capas de sistema**. Lo ordeno como ingeniero de producto de alto nivel, integrando lo de Gemini y lo que yo te propuse.

Fase 2 — SOFIAA Alpha Hardening

Objetivo general

Convertir el prototipo funcional en una **plataforma sólida, inteligente, medible y segura**, lista para despliegue público en Vercel sin perder elegancia, velocidad ni control.

Orden correcto de implementación

2.1 Congelación de arquitectura y deuda técnica

Antes de agregar nada, se congela la base actual.

Tareas:

- Documentar el estado exacto del sistema actual.
- Definir qué módulos ya son “core” y no se reescriben.
- Separar claramente UI, IA, memoria, analítica y seguridad.
- Crear un changelog técnico.
- Establecer convenciones de código, nombres y estados.

Entregable:

- ARCHITECTURE_FREEZE.md
- CODE_STANDARDS.md
- CHANGELOG_ALPHA.md

Criterio de salida:

- Nadie toca arquitectura base sin pasar por revisión.
-

2.2 Guardrails de contexto y seguridad activa

Esto es prioridad alta. Aquí entra fuerte lo de Gemini.

Tareas:

- Crear una capa de validación previa al endpoint `/api/chat`.
- Detectar prompt injection, solicitudes de secretos, fuga de contexto y abuso.
- Restringir acceso a frases clave, variables y rutas ocultas.
- Definir respuestas seguras cuando el input sea sospechoso.
- Implementar validación doble: cliente + servidor.

Entregable:

- `guardrails.engine.ts`
- `security.rules.ts`
- `safety.response.map.ts`

Criterio de salida:

- El sistema rechaza o neutraliza intentos de extracción de secretos sin romper la experiencia.
-

2.3 Telemetría automática viva

Aquí entra lo más valioso para tu marco doctoral y Lean Six Sigma.

Tareas:

- Crear `useSofiaaTelemetry.ts`.
- Medir tiempo hasta respuesta, clicks, retrocesos, pausas, errores, conversiones.
- Generar eventos estandarizados desde el frontend.
- Guardar todo en local de forma estructurada.
- Conectar el panel secreto con gráficos de sesión.

Entregable:

- `telemetry.events.ts`
- `useSofiaaTelemetry.ts`
- `metrics.schema.ts`

Criterio de salida:

- Cada sesión produce datos útiles para IFC, TTI, CKS, CVR e IAI.
-

2.4 Semantic Cache local

Esto reduce latencia, costo y hace que el sistema se sienta vivo.

Tareas:

- Cachear respuestas recurrentes por intención + contexto.
- Reutilizar respuestas ya validadas en la sesión.
- Guardar pares pregunta-respuesta en localStorage o IndexedDB.
- Agregar TTL y reglas de invalidación.
- Evitar llamadas innecesarias a la IA.

Entregable:

- semantic-cache.ts
- cache.policy.ts
- cache.adapter.ts

Criterio de salida:

- Preguntas repetidas se responden casi al instante sin volver a consumir modelo.
-

2.5 Goal Engine real

Esto transforma SOFIAA de chatbot a sistema operativo de experiencia.

Tareas:

- Definir objetivos base:
 - Informarse
 - Comparar
 - Aprender
 - Decidir
 - Contactar
 - Contratar
- Clasificar intención y convertirla en objetivo.
- Crear rutas de acompañamiento por objetivo.
- Definir salida esperada por cada objetivo.

Entregable:

- `goal.engine.ts`
- `intent.map.ts`
- `goal.routes.ts`

Criterio de salida:

- SOFIAA ya no responde solo “qué dijo”, sino “qué quiere lograr”.
-

2.6 Memory Timeline

Aquí entra mi propuesta para hacer el sistema más memorable y premium.

Tareas:

- Guardar historial semántico resumido.
- Mostrar una línea temporal de eventos importantes.
- Permitir retomar sesión desde puntos clave.
- Diferenciar memoria operativa, contextual y histórica.

Entregable:

- `memory.timeline.ts`
- `memory.summary.ts`
- `timeline.ui.tsx`

Criterio de salida:

- El usuario siente continuidad real, no solo un chat que reinicia.
-

2.7 Generative UI segura

Esto es la apuesta más disruptiva de Gemini, pero con control.

Tareas:

- Hacer que la IA no solo devuelva texto, sino también estructura.
- Usar salida estructurada tipo JSON schema.
- Limitar lo que la IA puede generar a un catálogo de componentes seguros.
- Renderizar cards, comparativas, botones, sliders, fechas, CTA.
- Nunca dejar que el modelo genere React libre.

Entregable:

- `ui.schema.ts`
- `response.renderer.ts`
- `component.registry.ts`

Criterio de salida:

- SOFIAA puede “construir interfaz” sin comprometer seguridad ni estabilidad.
-

2.8 Visual Intelligence Layer

Aquí entra lo mío: el sistema debe verse inteligente, no solo serlo.

Tareas:

- Definir estados visuales del Orbe.
- Vincular el estado visual con el estado cognitivo.
- Añadir respuesta visual a escucha, pensamiento, error y éxito.
- Construir el principio de Progressive Experience Disclosure.
- Mostrar solo lo necesario, cuando sea necesario.

Entregable:

- `orb.states.ts`
- `orb.controller.ts`
- `experience.disclosure.ts`

Criterio de salida:

- La UI “respira” y acompaña el momento cognitivo.
-

2.9 Knowledge Graph v0

Aquí ya empezamos a movernos a nivel Google.

Tareas:

- Convertir documentos y entidades en nodos.
- Relacionar usuario, objetivos, servicios, sesiones y respuestas.
- Permitir consultas por relación, no solo por texto.
- Empezar con una versión liviana, no enterprise.

Entregable:

- `graph.model.ts`
- `entity.extractor.ts`
- `knowledge.linker.ts`

Criterio de salida:

- SOFIAA empieza a entender relaciones, no solo documentos.
-

2.10 Micro-agentes limitados

Esto sí, pero con freno.

Tareas:

- No hacer 10 agentes.
- Crear solo 3 especialistas:
 - Research
 - Business
 - Creative
- El router decide cuál usar.
- Cada agente tiene un prompt corto y específico.

Entregable:

- `agent.router.ts`
- `agents/research.ts`
- `agents/business.ts`
- `agents/creative.ts`

Criterio de salida:

- El sistema delega sin volverse inmanejable.
-

2.11 Observabilidad y panel cognitivo

Esto amarra todo lo anterior.

Tareas:

- Mostrar sesiones, objetivos, latencia, fricción, éxito y abandono.
- Ver gráficas de telemetría en el panel secreto.

- Relacionar métricas con experiencia.

Entregable:

- `cognitive.dashboard.tsx`
- `analytics.pipeline.ts`
- `session.reports.ts`

Criterio de salida:

- El sistema no solo conversa: también se analiza solo.
-

2.12 Pre-deploy hardening

Antes de Vercel, se cierra todo.

Tareas:

- Variables de entorno.
- Manejo de errores.
- Timeouts.
- Rate limits.
- Fallbacks.
- Validación de producción.
- Pruebas de carga ligera.
- Limpieza de debug logs.

Entregable:

- `production.checklist.md`
- `deploy.ready.flag`

Criterio de salida:

- Se puede desplegar sin romper la experiencia.
-

El orden final de ejecución

Si lo reduces a una secuencia limpia, el orden correcto es este:

1. Congelar arquitectura.
2. Blindar seguridad.

3. Activar telemetría.
 4. Crear semantic cache.
 5. Construir goal engine.
 6. Agregar memory timeline.
 7. Integrar generative UI.
 8. Refinar capa visual inteligente.
 9. Montar knowledge graph v0.
 10. Limitar micro-agentes.
 11. Encender panel cognitivo.
 12. Cerrar hardening para Vercel.
-

Qué aporta cada bloque

Gemini aportó:

- Generative UI.
- Guardrails.
- Telemetría.
- Semantic cache.
- Micro-agentes.

Yo agregaría y ordenaría así:

- Goal Engine.
 - Memory Timeline.
 - Knowledge Graph v0.
 - Visual Intelligence Layer.
 - Observabilidad cognitiva.
 - Pre-deploy hardening.
-

Regla de ingeniería para esta fase

Una mejora funcional por PR.

Una capa por sprint.

Nada de sobrecargar el núcleo.

SOFIAA ya existe.

Ahora toca volverlo:

- más rápido,

- más seguro,
- más inteligente,
- más visible,
- más medible,
- más difícil de copiar.

Si ejecutas esta fase en ese orden, el siguiente salto ya no será “otro beta”.
Será un sistema que se siente como una categoría nueva.